

رسالة محمد



یادگیری ماشین

مدرس: محمدرضا محمدی

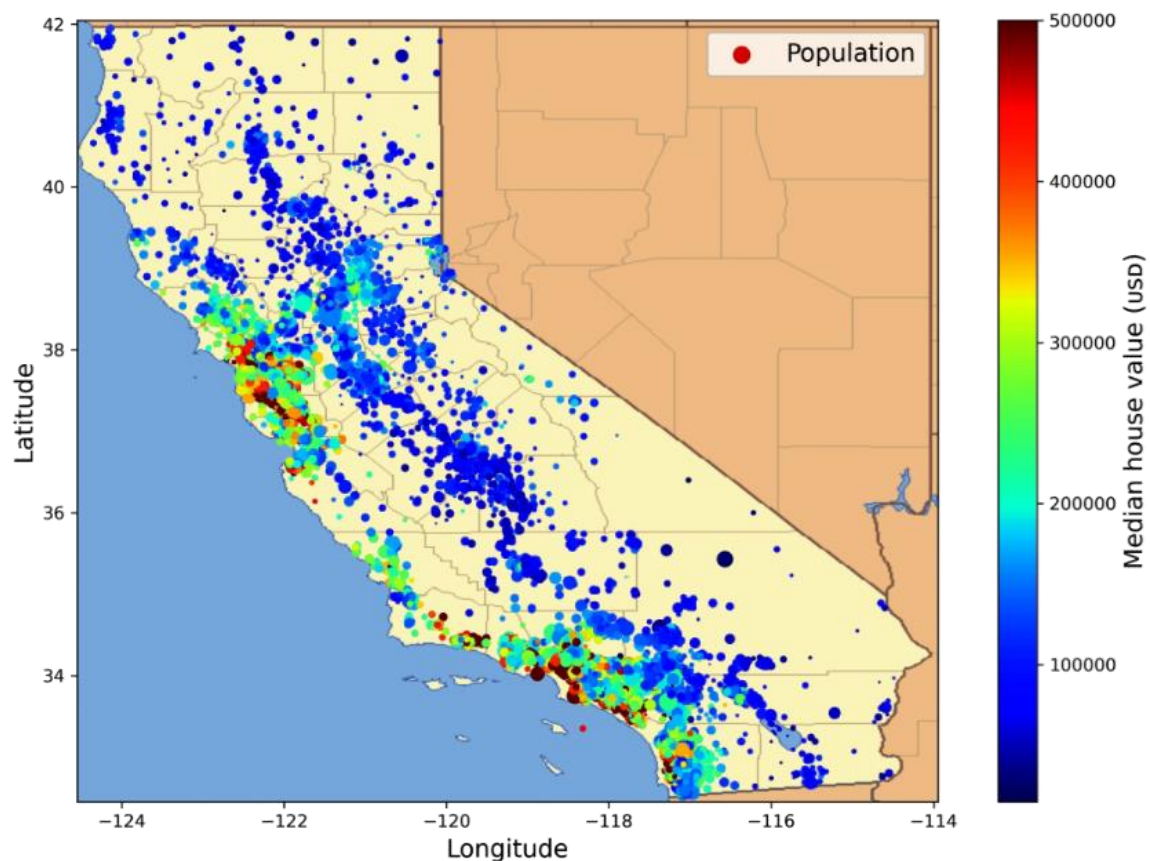
زمستان ۱۴۰۱

پروژه یادگیری ماشین انتها به انتها

End-to-End Machine Learning Project

پروژه یادگیری ماشین انتها به انتها

- می‌خواهیم با یک مثال ساختگی (تخمین قیمت مسکن) مراحل اصلی یک پروژه یادگیری ماشین را بررسی کنیم



- نگاه به تصویر بزرگ (big picture)
- دریافت داده‌ها
- بررسی و مصورسازی داده‌ها
- آماده‌سازی داده‌ها برای الگوریتم‌های یادگیری ماشین
- انتخاب مدل و آموزش آن
- تنظیم دقیق مدل
- ارائه راه‌حل پیشنهادی
- راه‌اندازی، نظارت و نگهداری سیستم

آموزش رگرسیون خطی

```
from sklearn.linear_model import LinearRegression
```

```
lin_reg = make_pipeline(preprocessing, LinearRegression())  
lin_reg.fit(housing, housing_labels)
```

```
>>> housing_predictions = lin_reg.predict(housing)  
>>> housing_predictions[:5].round(-2) # -2 = rounded to tl  
array([243700., 372400., 128800., 94400., 328300.])  
>>> housing_labels.iloc[:5].values  
array([458300., 483800., 101700., 96100., 361800.])
```

```
>>> from sklearn.metrics import mean_squared_error  
>>> lin_rmse = mean_squared_error(housing_labels, housing_predictions,  
...                               squared=False)  
...  
...  
>>> lin_rmse  
68687.89176589991
```

• نتیجه چندان خوب نیست!

- در شرایطی که اکثر محله‌ها دارای قیمت در بازه 120,000\$ و 265,000\$ هستند، خطای پیش‌بینی 68,687\$ زیاد است
- مدل underfit شده است
- می‌توان از مدل قوی‌تر و/یا ویژگی‌های بهتر استفاده کرد

مدل رگرسیون درخت تصمیم

- درخت تصمیم مدل پیچیده‌تری است که فضا را به چندین بخش تقسیم می‌کند و می‌تواند روابط پیچیده غیرخطی را بیابد
- روی داده‌های آموزشی بیش از حد خوب است! احتمالا overfit شده است
 - روی داده‌های اعتبارسنجی باید ارزیابی کنیم

```
from sklearn.tree import DecisionTreeRegressor
```

```
tree_reg = make_pipeline(preprocessing, DecisionTreeRegressor(random_state=42))  
tree_reg.fit(housing, housing_labels)
```

```
>>> housing_predictions = tree_reg.predict(housing)  
>>> tree_rmse = mean_squared_error(housing_labels, housing_predictions,  
...                               squared=False)  
...  
...  
>>> tree_rmse  
0.0
```

ارزیابی متقابل

- ارزیابی متقابل k-fold به راحتی در sklearn قابل انجام است

```
from sklearn.model_selection import cross_val_score

tree_rmse = -cross_val_score(tree_reg, housing, housing_labels,
                              scoring="neg_root_mean_squared_error", cv=10)
```

```
>>> pd.Series(tree_rmse).describe()
```

```
count      10.000000
mean      66868.027288
std       2060.966425
min       63649.536493
25%      65338.078316
50%      66801.953094
75%      68229.934454
max       70094.778246
dtype: float64
```

- مدل درخت تصمیم انتخاب شده برای این مجموعه داده عملکرد مطلوبی ندارد و در حد همان رگرسیون خطی است
- البته overfit شده است

مدل رگرسیون جنگل تصادفی

- در این روش، چندین درخت تصمیم با زیرمجموعه‌های تصادفی از ویژگی‌ها آموزش داده می‌شود و پیش‌بینی آنها میانگین‌گیری می‌شود

```
from sklearn.ensemble import RandomForestRegressor
```

```
forest_reg = make_pipeline(preprocessing,  
                            RandomForestRegressor(random_state=42))
```

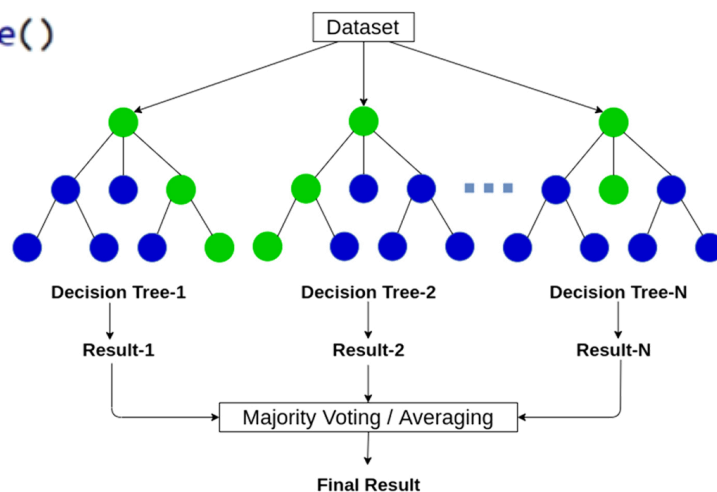
```
forest_rmse = cross_val_score(forest_reg, housing, housing_labels,  
                              scoring="neg_root_mean_squared_error", cv=10)
```

ensembles -

boosting -

```
>>> pd.Series(forest_rmse).describe()
```

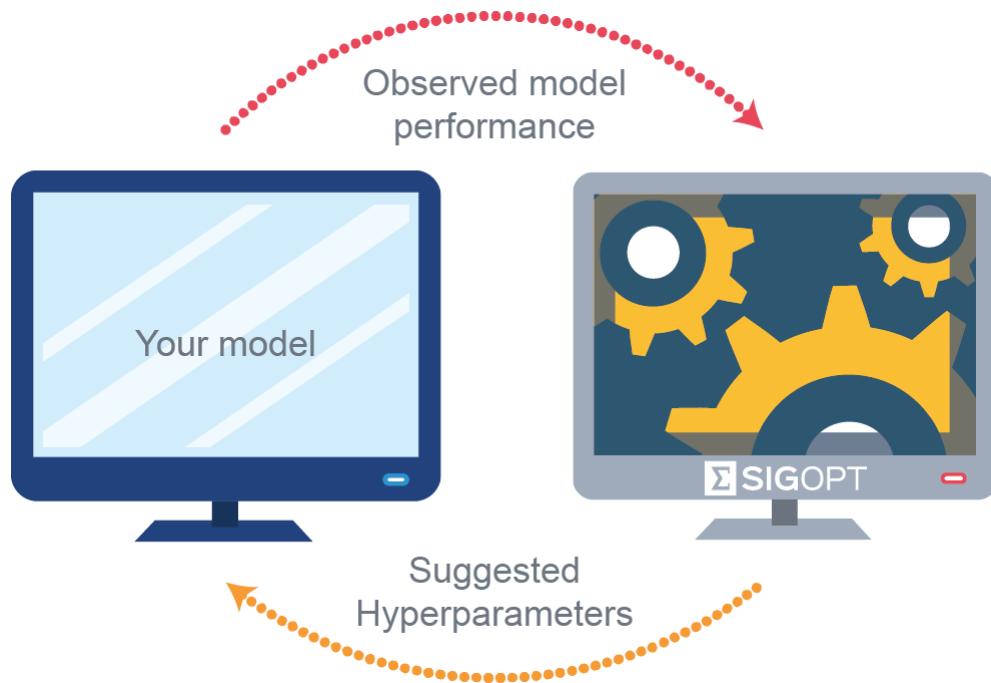
count	10.000000
mean	47019.561281
std	1033.957120
min	45458.112527
25%	46464.031184
50%	46967.596354
75%	47325.694987
max	49243.765795
dtype:	float64



- با وجود بهبود بدست آمده، روی داده‌های آموزشی خطا \$17,474\$ است که نشان می‌دهد هنوز تا حدی overfit شده است

تنظیم دقیق مدل

- هر کدام از مدل‌ها دارای آبرپارامترهای متعددی است که انتخاب مقدار مناسب برای آنها تاثیر زیادی در عملکرد مدل دارد
- تنظیم دستی این آبرپارامترها به صورت مناسب نیاز به تجربه و دانش زیادی دارد
- می‌توان این تنظیم را به صورت خودکار انجام داد



جستجوی شبکه‌ای (Grid)

```
from sklearn.model_selection import GridSearchCV

full_pipeline = Pipeline([
    ("preprocessing", preprocessing),
    ("random_forest", RandomForestRegressor(random_state=42)),
])

param_grid = [
    {'preprocessing__geo__n_clusters': [5, 8, 10],
     'random_forest__max_features': [4, 6, 8]},
    {'preprocessing__geo__n_clusters': [10, 15],
     'random_forest__max_features': [6, 8, 10]},
]

grid_search = GridSearchCV(full_pipeline, param_grid, cv=3,
                           scoring='neg_root_mean_squared_error')
grid_search.fit(housing, housing_labels)
```

	n_clusters	max_features	split0	split1	split2	mean_test_rmse
12	15	6	43460	43919	44748	44042
13	15	8	44132	44075	45010	44406
14	15	10	44374	44286	45316	44659
7	10	6	44683	44655	45657	44999
9	10	6	44683	44655	45657	44999

- ساده‌ترین روش جستجو این است که برای هر ابرپارامتر چند مقدار را در نظر بگیریم و همه ترکیب‌های ممکن را ارزیابی کنیم
- بر اساس __ در رشته، نام بخش و متغیر مورد نظر برای جستجو مشخص می‌شود
- در مجموع $2 \times 3 + 3 \times 3$ ترکیب مختلف ارزیابی می‌شود

- ۴۵ بار مدل آموزش داده می‌شود

- می‌توان تعداد خوشه‌های بیشتر را بررسی کرد

جستجوی تصادفی (Randomized)

- اگر تعداد ترکیب‌های مورد جستجوی کم نباشد و به خصوص برخی از ابرپارامترها دارای تاثیر زیادی نباشند، جستجوی تصادفی می‌تواند مناسب‌تر باشد
- برای هر ابرپارامتر یک توزیع تصادفی تعریف می‌شود که در هر تکرار، یک مقدار تصادفی از آن انتخاب می‌شود

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distributions = {'preprocessing__geo__n_clusters': randint(low=3, high=50),
                       'random_forest__max_features': randint(low=2, high=20)}

rnd_search = RandomizedSearchCV(
    full_pipeline, param_distributions=param_distributions, n_iter=10, cv=3,
    scoring='neg_root_mean_squared_error', random_state=42)

rnd_search.fit(housing, housing_labels)
```

راه‌اندازی، نظارت و نگهداری سیستم

- برای راه‌اندازی سیستم نیاز است بهترین مدلی را که آموزش داده‌اید ذخیره کنید، فایل را به محیط استفاده منتقل کنید و آن را بارگذاری کنید

```
import joblib
```

```
joblib.dump(final_model, "my_california_housing_model.pkl")
```

```
import joblib
```

```
[...] # import KMeans, BaseEstimator, TransformerMixin, rbf_kernel, etc.
```

```
def column_ratio(X): [...]
```

```
def ratio_name(function_transformer, feature_names_in): [...]
```

```
class ClusterSimilarity(BaseEstimator, TransformerMixin): [...]
```

```
final_model_reloaded = joblib.load("my_california_housing_model.pkl")
```

```
new_data = [...] # some new districts to make predictions for  
predictions = final_model_reloaded.predict(new_data)
```

راه‌اندازی، نظارت و نگهداری سیستم

- به عنوان مثال، می‌توان از این مدل در یک برنامه تحت وب استفاده کرد
 - می‌توان از سیستم‌های محاسبات ابری مانند Google Cloud AI Platform استفاده کرد
- پس از توسعه، نیاز است تا کدی برای نظارت مستمر بر عملکرد سیستم نوشته شود
 - ممکن است کد به دلیل خطا متوقف شده باشد
 - ممکن است عملکرد آن در طول زمان کاهش بیابد
- مثلاً اگر کد مربوط به سیستم پیشنهاددهنده (Recommender) است، و کاربران از آن کمتر استفاده می‌کنند
- می‌توان نمونه‌هایی را برای ناظر انسانی فرستاد و از بازخورد آن کمک گرفت



راه‌اندازی، نظارت و نگهداری سیستم

- اگر داده‌ها تغییر می‌کنند، باید مجموعه داده‌ها را مرتباً به روز کرد و مدل را مجدداً آموزش داد
- لازم است کل فرآیند به‌روزرسانی را تا حد امکان خودکار کرد
 - جمع‌آوری داده‌های جدید و برچسب‌گذاری آنها
 - توسعه برنامه‌ای برای آموزش مدل و تنظیم خودکار ابرپارامترها
 - برای مثال هر روز یا هر هفته اجرا شود
 - توسعه برنامه‌ای برای ارزیابی و مقایسه مدل‌ها و جایگزینی مدل جدید در صورت بهبود
 - حتماً از داده‌ها و مدل‌ها باید پشتیبان‌گیری شود

دسته‌بندی

Classification

مجموعه داده MNIST

- در این فصل از مجموعه داده MNIST استفاده می‌کنیم که شامل ۷۰۰۰۰ نمونه تصویر کوچک از رقم‌های دستنویس است



```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)

>>> X, y = mnist.data, mnist.target

>>> X.shape
(70000, 784)

>>> y.shape
(70000,)

>>> y
array(['5', '0', '4', ..., '4', '5', '6'], dtype=object)
```

مجموعه داده MNIST

```
import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")
```

```
some_digit = X[0]
plot_digit(some_digit)
plt.show()
```



```
>>> y[0]
'5'
```

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

- برای هر داده ۷۸۴ ویژگی وجود دارد که شدت روشنایی 28×28 پیکسل مربوط به آن است
- می‌توان هر نمونه را به 28×28 تغییر شکل داد
- بخشی را برای آزمون جدا می‌کنیم:

دسته‌بندی دو کلاسه

- فرض کنید می‌خواهیم یک دسته‌بند برای تشخیص رقم ۵ آموزش بدهیم ("5-detector")

```
y_train_5 = (y_train == '5') # True for all 5s, False for all other digits  
y_test_5 = (y_test == '5')
```

- برای شروع از دسته‌بند خطی SGD استفاده می‌کنیم

```
from sklearn.linear_model import SGDClassifier
```

```
sgd_clf = SGDClassifier(random_state=42)  
sgd_clf.fit(X_train, y_train_5)
```

```
>>> sgd_clf.predict([some_digit])  
array([ True])
```

معیارهای ارزیابی

- دقت (accuracy) یکی از متداول ترین معیارهای ارزیابی در مسئله های دسته بندی است

```
>>> from sklearn.model_selection import cross_val_score
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.95035, 0.96035, 0.9604 ])
```

- به نظر دقت خوبی است!

- اما در این مثال بیش از ۹۰٪ از نمونه های متعلق به دسته ۰ (رقم غیر از ۵) بوده اند و اگر دسته بند همه نمونه ها را به عنوان دسته ۰ دسته بندی کند، بالای ۹۰٪ دقت خواهد داشت!
- دقت معیار خوبی نیست برای مسائلی که دسته های آن متوازن نیستند

ماتریس درهم‌ریختگی (Confusion)

- یک ماتریس که درایه ij آن برابر با تعداد نمونه‌هایی است که متعلق به دسته i بوده‌اند و به عنوان دسته j شناسایی شده‌اند

- هر سطر نمونه‌های یک دسته واقعی را نشان می‌دهد (actual class)

- هر ستون نمونه‌های پیش‌بینی شده برای یک دسته را نشان می‌دهد (predicted class)

```
from sklearn.model_selection import cross_val_predict
```

```
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)
```

```
>>> from sklearn.metrics import confusion_matrix
>>> cm = confusion_matrix(y_train_5, y_train_pred)
>>> cm
array([[53892,   687],
       [ 1891, 3530]])
```

		Predicted Class	
		Negative	Positive
Actual Class	Negative	TN	FP
	Positive	FN	TP

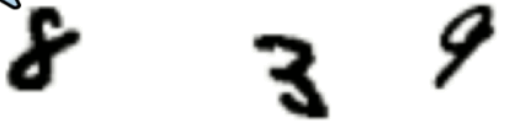
ماتریس درهم ریختگی (Confusion)

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

• صحت (precision): دقت پیش بینی های مثبت

• پوشش یا بازیابی (recall): درصد نمونه های مثبتی که به درستی دسته بندی شده است

		Predicted	
		Negative	Positive
Actual	Negative	TN 	FP 
	Positive		TP 
		FN	Precision (e.g., 3 out of 4)
		Recall (e.g., 3 out of 5)	

		Predicted Class	
		Negative	Positive
Actual Class	Negative	TN	FP
	Positive	FN	TP

```
array([[53892, 687],
       [1891, 3530]])
```

معیارهای ارزیابی

```
>>> from sklearn.metrics import precision_score, recall_score
>>> precision_score(y_train_5, y_train_pred) # == 3530 / (687 + 3530)
0.8370879772350012
>>> recall_score(y_train_5, y_train_pred) # == 3530 / (1891 + 3530)
0.6511713705958311
```

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

```
>>> from sklearn.metrics import f1_score
>>> f1_score(y_train_5, y_train_pred)
0.7325171197343846
```

```
array([[53892, 687],
       [1891, 3530]])
```

- هر کدام از این دو معیار فقط از یک زاویه به عملکرد مدل نگاه می‌کنند

- می‌توان مقادیر آنها را ترکیب کرد تا بتوان با یک مقدار عددی، الگوریتم‌های مختلف را مقایسه کرد

- بسته به مسئله، می‌توان اهمیت precision یا recall را افزایش داد